

CS 236: Operating Systems Lab

Spring 2025-26

Labquiz1

Instructions

- No Internet. No phone. No devices. No handwritten notes. No books.
- No friends (strictly for the duration of the exam only!).
- Unethical practices will be an automatic recommendation for the FR grade.
- **Hardcoding of outputs will result in -5 marks per instance.**
- Please read the questions and submission instructions carefully.
- The labquiz consists of 3 questions, each with a few sub-questions.
- **All problems are compulsory.**

Submission Guidelines

- For submission, create a folder on the Desktop named: **submission_<rollno>**
E.g., a directory named **submission_24m0761**
- **Please strictly adhere to this format otherwise, your submission will not count.**
- Add all the submission files (mentioned below) in the aforementioned folder. Your folder structure should look something like this:

```
submission_<rollno>/
├── part1
│   ├── p1a.c
│   └── p1b.c
├── part2
│   ├── p2a.c
│   └── p2b.c
└── part3
    ├── p3a.c
    ├── p3b.c
    └── p3c.c
```

- **To submit —**
 1. Execute the **check** command on the terminal.
To ensure that the submission folder exists, is not empty and create the tar file.
 2. Execute the **submit** command on the terminal to submit.
TA will input the password to complete the submission.
 3. Ensure Step 2 to confirm submission.
- Note: The **check** and **submit** programs are pre-installed.

1. **my_max = max (my_max, your_max)** (5 marks)

Find the maximum value in an integer array (of size N) of randomly generated numbers. The task is accomplished by N child processes, each of which examines a single value in the array and the maximum value returned by its child process.

The **ith** child process accesses the **ith** index in the array and compares this value with that returned by its child process to return its estimated maximum value. The child process accessing the **N-1th** index returns its own value as its estimated maximum.

The main parent creates child process 0.

- **child process 0 (access index 0 of the array):** Creates **child process 1**.
 - **child process 1 (access index 1 of the array):** Creates **child process 2**.
 - ... and so on until **Child Process N-1**.
- a. The template code **p1a.c** takes as input the number N and creates an array with random numbers. Extend the program **p1a.c** such that each process (starting from the main parent process) creates **exactly ONE child** (except the last which creates none) until the depth N is reached. Each child process must print its process id, index, and value of array at that index. Use the print statements provided in the template. (2 marks)

Sample Output

```
Shell
> ./p1a 3
[pid 81302] Created array: 66 68 53
[pid 81302] Created child process: 81303
[pid 81303] arr[0]: 66, Created child process: 81304
[pid 81304] arr[1]: 68, Created child process: 81305
[pid 81305] arr[2]: 53
```

- b. Extend **p1a.c** to **p1b.c**, to propagate the maximum value estimate of each child process back to its parent process. Note that the maximum value of each child process depends on the value it accesses and the estimate of its child process.

The final result must be printed only by the original parent process. (3 marks)

Hint: The **exit** system call can do more than just exit a process.

NOTE:

Array values are restricted to [0, 99] in the template code, safe for exit status (0-255).

Sample Output

```
Shell
> ./p1b 3
[pid 82625] Created array: 15 33 32
[pid 82625] Created child process: 82626
[pid 82626] arr[0]: 15, Created child process: 82627
[pid 82627] arr[1]: 33, Created child process: 82628
[pid 82628] arr[2]: 32
[pid 82628] arr[2]: 32, max_est: 32
[pid 82627] arr[1]: 33, child_max_est: 32, max_est: 33
[pid 82626] arr[0]: 15, child_max_est: 33, max_est: 33
[pid 82625] max(array): 33
```

2. sending mixed signals! (6 marks)

- a. Write a program **p2a.c** which creates a child process using fork and registers a different signal handler for the signal **SIGUSR1** for both parent and child processes. Use the provided auxiliary program **send_siguser1** to verify that your process responds correctly to external signals. Use the print statements provided in the template. Also print the number of times each signal handler was invoked. Note that the parent and child processes need to run forever to keep receiving signals. **(3 marks)**

Sample Output

Terminal 1

```
Shell
> ./p2a
[pid 85376]: Parent Process
[pid 85377]: Child Process
[pid 85376]: Parent Received Signal 1
[pid 85377]: Child Received Signal 1
[pid 85376]: Parent Received Signal 2
^C
```

Terminal 2

```
Shell
> ./send_siguser1 85376
Signal 10 sent to process 85376
> ./send_siguser1 85377
Signal 10 sent to process 85377
> ./send_siguser1 85376
Signal 10 sent to process 85376
```

- b. Extend your **p2a.c** to **p2b.c**, this program accepts a single **command-line argument N**. The signal handler of the parent process sends a **SIGUSER1** signal to the child process. The child signal handler responds with a **SIGUSER1** signal to the parent process, the parent process responds with a **SIGUSER1** signal to the child process, and the child process responds with a **SIGUSER1** signal and the sequence continues. The alternative invocation of the parent and child signal handlers should happen for **N** times.

Refer to the sample output for an example execution run. After alternating **N** signals handled by each process the processes should terminate. **(3 marks)**

Sample Output

Terminal 1

```
Shell
./p2b 4
[pid 166006]: Parent process
[pid 166007]: Child process
[pid 166006]: Parent Received Signal 1
[pid 166007]: Child Received Signal 1
[pid 166006]: Parent Received Signal 2
[pid 166007]: Child Received Signal 2
[pid 166006]: Parent Received Signal 3
[pid 166007]: Child Received Signal 3
[pid 166006]: Parent Received Signal 4
[pid 166007]: Child Received Signal 4
```

Terminal 2

(send the first signal to one of the processes to get the sequence started)

```
Shell
./send_siguser1 166006
Signal 10 sent to process 166006
```

3. piped (8 marks)

For each part in this problem write a C program that takes the path to a directory as a command-line argument.

- a. Write a C program **p3a.c** that creates a child process to execute the **ls command** for the given directory path. Hint: Look up usage of the **exec1p** system call. **(1 mark)**.

```
Shell
> ./p3a /
bin                lib                mnt                snap
bin.usr-is-merged  lib32              opt                srv
boot              lib64              proc               swap.img
cdrom             lib.usr-is-merged  root               sys
dev              libx32             run                tmp
etc              lost+found         sbin               usr
home            media             sbin.usr-is-merged var

> ./p3a /sys
block  class  devices  fs          kernel  power
bus    dev    firmware hypervisor  module
```

- b. Extend your **p3a.c** to **p3b.c** so that the output of **ls** is redirected to a file named **output.txt** using the **dup2()** system call.

NOTE: You must **NOT** use **read()** or **write()** system calls to move the data. **(3 marks)**

```
Shell
> ./p3b /sys
> cat output.txt
block
class
devices
fs
kernel
power
```

```
bus
dev
firmware
hypervisor
module
```

- c. Extend your **p3b.c** to **p3c.c** to implement a pipeline equivalent to **ls <dir> | wc -l**. Redirect the output of **ls <dir>** to another process that executes **wc -l** to count the number of entries in that directory. (4 marks)

Hint:

1. Fork two child processes. One to execute **ls**, and another to execute **wc -l**
2. Use **dup2** to redirect the input/output between these child processes.
3. Make sure you close all the unused pipe ends after **fork** and before **exec**

Sample Output

```
Shell
./p3c /
28
> ls / | wc -l
28
> ./p3c /sys/
11
> ls /sys/ | wc -l
11
> ./p3c .
18
> ls . | wc -l
18
```